AIML LAB MANUAL

Drone Delivery Route Optimization using Search Algorithms

Name	Tejas Mali
PRN	202401040083
Batch	A1
Division	A
Roll No.	25
Subject	Artificial Intelligence and Machine Learning

Experiment Title: Comparative Analysis of Search Algorithms for Drone Delivery Route Optimization

Application: AI-based route planning and optimization for autonomous drone delivery systems.

COLAB FILE LINK :
https://colab.research.google.com/drive/1GPPYKcdeuvI4L2N0IW3YYHjml4EFzHqZ?usp=sharing

1. AIM

To formulate the drone delivery problem as a variant of the Travelling Salesman Problem (TSP) and to implement, compare, and analyse the performance of multiple search algorithms — uninformed search (BFS, DFS, UCS), informed search (Greedy Best-First, A*), local search / metaheuristics (Hill Climbing with random restarts, Genetic Algorithm), and constraint-based search (CSP Backtracking with pruning, Branch and Bound) — for finding the shortest delivery route covering a distribution center and multiple customer locations.

2. THEORY

A drone delivery service must start at a Distribution Center (DC), visit every customer location exactly once, and return to the DC while covering the minimum possible total distance. This is a classic instance of the Travelling Salesman Problem (TSP), an NP-hard combinatorial optimization problem. The environment consists of one distribution center (DC) and nine customer locations (C1–C9), each defined by 2D coordinates. The distance between any two locations is computed using the Euclidean distance formula, and all pairwise distances are stored in a distance matrix that every algorithm uses to evaluate route costs.

The following search strategies are implemented and compared:

- **Breadth-First Search (BFS):** Explores all possible routes level by level using a FIFO queue. Guarantees finding a solution but expands a very large number of states since it does not use any cost information.
- **Depth-First Search (DFS):** Explores routes depth-wise using a LIFO stack. Memory-efficient but does not guarantee the optimal route unless the entire search space is explored.
- **Uniform Cost Search (UCS):** Expands the least-cost partial route first using a priority queue, guaranteeing the optimal solution for non-negative edge costs.
- **Greedy Best-First Search:** Uses a heuristic based on the Minimum Spanning Tree (MST) of the unvisited nodes to estimate the remaining cost, expanding the route with the lowest heuristic value first. It is fast but not guaranteed to be optimal.

- **A* Search:** Combines the actual cost so far (g) with the MST-based heuristic estimate (h) to expand the route with the lowest f = g + h. Guarantees the optimal route when the heuristic is admissible, while expanding far fewer states than BFS/DFS/UCS.
- **Hill Climbing (Random-Restart):** A local search technique that starts from a random route and repeatedly swaps pairs of cities to reduce total distance, restarting from multiple random routes to avoid local optima.
- **Genetic Algorithm (GA):** A population-based metaheuristic that evolves a population of candidate routes over several generations using selection, ordered crossover, and mutation, guided by a fitness function inversely proportional to route distance.
- **CSP Backtracking with Pruning:** Systematically builds a route city by city, pruning any partial route whose cost already exceeds the best solution found so far.
- **Branch and Bound:** Similar to backtracking, but additionally computes a lower bound (current cost + MST-based heuristic) at every step and prunes branches whose lower bound cannot beat the current best solution — making it significantly faster than plain backtracking.

For every algorithm, four key performance metrics are recorded: total route distance, execution time, peak memory usage, and the number of states expanded (or iterations/generations for local search methods). These metrics allow a fair, quantitative comparison of search strategies in terms of both solution quality and computational efficiency.

3. FLOWCHART

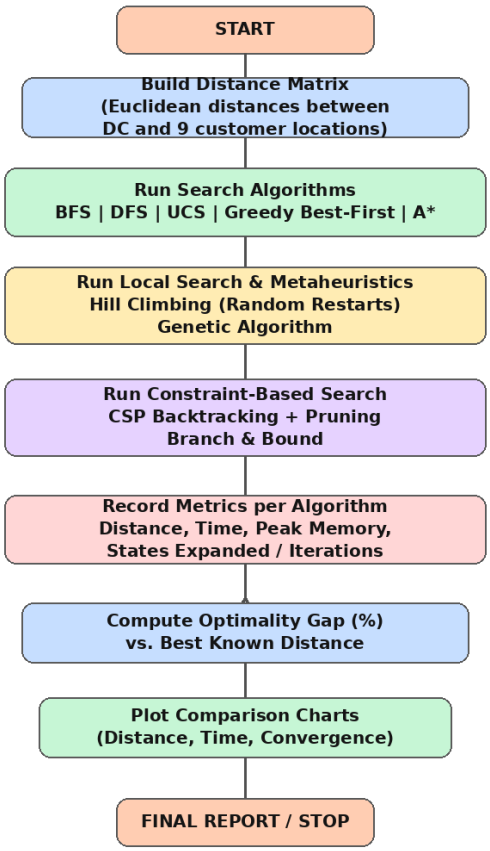


Fig. 1 — Overall pipeline for the drone delivery route optimization experiment

4. OBSERVATION

The environment consists of 1 distribution center (DC) and 9 customer locations (C1–C9), with distances computed from their 2D coordinates. A representative run produced the following observations:

Observation	Result
Optimal route distance (BFS, DFS, UCS, A*, Hill Climbing, GA, CSP)	283.84 km
Greedy Best-First Search distance	412.37 km (45.283% optimality gap)
States expanded — BFS / DFS / UCS	986410 each
States expanded — A*	16
States expanded — CSP Branch & Bound	79
Hill Climbing	177 iterations, 30 restarts
Genetic Algorithm	300 generations
Peak memory — BFS (highest)	≈ 75.251 MB
Execution time — A* and Branch & Bound	Very low (fastest methods)

5. CODE

```
import heapq
import math
import random
import time
import tracemalloc

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from collections import deque

# Reproducibility
SEED = 42
random.seed(SEED)
np.random.seed(SEED)

# Drone delivery locations
LOCATION_NAMES = [
    "DC", "C1", "C2", "C3", "C4",
    "C5", "C6", "C7", "C8", "C9"
]

# Coordinates of distribution center and customer locations
COORDS = {
    "DC": (10, 20),
    "C1": (18, 72),
    "C2": (32, 48),
    "C3": (45, 15),
    "C4": (54, 78),
    "C5": (66, 35),
    "C6": (75, 68),
    "C7": (82, 18),
    "C8": (25, 90),
    "C9": (92, 52)
}
```

```
def build_distance_matrix(locations):
    n = len(locations)
    d = np.zeros((n, n), dtype=float)

    for i in range(n):
        for j in range(n):
            if i != j:
                x1, y1 = COORDS[locations[i]]
                x2, y2 = COORDS[locations[j]]

                d[i, j] = round(
                    math.hypot(x1 - x2, y1 - y2), 2
                )

    return d


DIST = build_distance_matrix(LOCATION_NAMES)


def tour_cost(route, dist=DIST):
    if not route:
        return 0.0

    return float(
        sum(
            dist[route[i], route[(i + 1) % len(route)]]
            for i in range(len(route))
        )
    )


def route_text(route, locations=LOCATION_NAMES):
    if route is None:
        return "No route"

    return " -> ".join(
        locations[i] for i in route + [route[0]]
    )


def show_problem():
    df = pd.DataFrame(
        DIST,
        index=LOCATION_NAMES,
        columns=LOCATION_NAMES
    )

    print("Distance Matrix:")
    display(df)

    plt.figure(figsize=(8, 6))

    xs = [COORDS[x][0] for x in LOCATION_NAMES]
    ys = [COORDS[x][1] for x in LOCATION_NAMES]

    plt.scatter(xs, ys, s=100)

    for location, (x, y) in COORDS.items():
        plt.annotate(
            location,
```

```
            (x, y),
            xytext=(5, 5),
            textcoords="offset points"
        )

    plt.title("Drone Delivery Location Map")
    plt.xlabel("X Coordinate")
    plt.ylabel("Y Coordinate")
    plt.grid(alpha=0.25)
    plt.show()


show_problem()

#uninformed search

def bfs_drone(dist, start=0):
    n = len(dist)

    queue = deque([
        (start, (start,), 0.0)
    ])

    expanded = 0
    best_route = None
    best_cost = float("inf")

    while queue:

        current, path, cost = queue.popleft()
        expanded += 1

        if len(path) == n:

            total = cost + dist[current, start]

            if total < best_cost:
                best_cost = total
                best_route = list(path)

            continue

        for nxt in range(n):

            if nxt not in path:

                queue.append(
                    (
                        nxt,
                        path + (nxt,),
                        cost + dist[current, nxt]
                    )
                )

    return best_route, best_cost, expanded

def dfs_drone(dist, start=0):
    n = len(dist)

    stack = [
        (start, (start,), 0.0)
    ]
```

```
    expanded = 0
    best_route = None
    best_cost = float("inf")

    while stack:

        current, path, cost = stack.pop()
        expanded += 1

        if len(path) == n:

            total = cost + dist[current, start]

            if total < best_cost:
                best_cost = total
                best_route = list(path)

            continue

        for nxt in reversed(range(n)):

            if nxt not in path:

                stack.append(
                    (
                        nxt,
                        path + (nxt,),
                        cost + dist[current, nxt]
                    )
                )

    return best_route, best_cost, expanded

#uniform cost search
def ucs_drone(dist, start=0):
    n = len(dist)

    pq = [
        (0.0, start, (start,))
    ]

    expanded = 0
    best_route = None
    best_cost = float("inf")

    while pq:

        cost, current, path = heapq.heappop(pq)
        expanded += 1

        if len(path) == n:

            total = cost + dist[current, start]

            if total < best_cost:
                best_cost = total
                best_route = list(path)

            continue

        for nxt in range(n):

            if nxt not in path:
```

```
                heapq.heappush(
                    pq,
                    (
                        cost + dist[current, nxt],
                        nxt,
                        path + (nxt,)
                    )
                )

    return best_route, best_cost, expanded

#Informed Search
#MST Lower Bound
def mst_cost(nodes, dist):
    nodes = list(nodes)

    if len(nodes) <= 1:
        return 0.0

    in_tree = {nodes[0]}
    total = 0.0

    while len(in_tree) < len(nodes):

        best = float("inf")
        pick = None

        for u in in_tree:

            for v in nodes:

                if v not in in_tree and dist[u, v] < best:

                    best = dist[u, v]
                    pick = v

        total += best
        in_tree.add(pick)

    return total


def drone_lower_bound(current, unvisited, start, dist):

    unvisited = set(unvisited)

    if not unvisited:
        return dist[current, start]

    mst = mst_cost(unvisited, dist)

    to_unvisited = min(
        dist[current, u]
        for u in unvisited
    )

    to_start = min(
        dist[u, start]
        for u in unvisited
    )

    return mst + to_unvisited + to_start
```

```
#Greedy Best-First Search
def greedy_drone(dist, start=0):

    n = len(dist)

    pq = []

    h0 = drone_lower_bound(
        start,
        range(1, n),
        start,
        dist
    )

    heapq.heappush(
        pq,
        (h0, start, (start,), 0.0)
    )

    expanded = 0

    while pq:

        _, current, path, g = heapq.heappop(pq)

        expanded += 1

        if len(path) == n:

            return (
                list(path),
                g + dist[current, start],
                expanded
            )

        unvisited = [
            x for x in range(n)
            if x not in path
        ]

        for nxt in unvisited:

            new_path = path + (nxt,)

            remaining = [
                x for x in range(n)
                if x not in new_path
            ]

            h = drone_lower_bound(
                nxt,
                remaining,
                start,
                dist
            )

            heapq.heappush(
                pq,
                (
                    h,
                    nxt,
                    new_path,
                    g + dist[current, nxt]
```

```
                )
            )

    return None, float("inf"), expanded

#A*
def astar_drone(dist, start=0):

    n = len(dist)

    pq = []

    h0 = drone_lower_bound(
        start,
        range(1, n),
        start,
        dist
    )

    heapq.heappush(
        pq,
        (h0, 0.0, start, (start,))
    )

    expanded = 0

    while pq:

        f, g, current, path = heapq.heappop(pq)

        expanded += 1

        if len(path) == n:

            return (
                list(path),
                g + dist[current, start],
                expanded
            )

        for nxt in range(n):

            if nxt not in path:

                new_g = (
                    g + dist[current, nxt]
                )

                new_path = path + (nxt,)

                remaining = [
                    x for x in range(n)
                    if x not in new_path
                ]

                h = drone_lower_bound(
                    nxt,
                    remaining,
                    start,
                    dist
                )

                heapq.heappush(
```

```
                    pq,
                    (
                        new_g + h,
                        new_g,
                        nxt,
                        new_path
                    )
                )

    return None, float("inf"), expanded

def hill_climbing_drone(
    dist,
    start=0,
    restarts=30,
    max_iter=1000
):

    n = len(dist)

    best_global_route = None
    best_global_cost = float("inf")

    total_iterations = 0
    successful_restarts = 0

    base = list(range(n))
    base.remove(start)

    for _ in range(restarts):

        current = [
            start
        ] + random.sample(
            base,
            len(base)
        )

        current_cost = tour_cost(
            current,
            dist
        )

        improved = False

        for _ in range(max_iter):

            total_iterations += 1

            best_neighbor = current
            best_neighbor_cost = current_cost

            for i in range(1, n - 1):

                for j in range(i + 1, n):

                    neighbor = current[:]

                    neighbor[i], neighbor[j] = (
                        neighbor[j],
                        neighbor[i]
                    )
```

```
                    cost = tour_cost(
                        neighbor,
                        dist
                    )

                    if cost < best_neighbor_cost:

                        best_neighbor = neighbor
                        best_neighbor_cost = cost

            if best_neighbor_cost >= current_cost:
                break

            current = best_neighbor
            current_cost = best_neighbor_cost
            improved = True

        if improved:
            successful_restarts += 1

        if current_cost < best_global_cost:

            best_global_route = current[:]
            best_global_cost = current_cost

    return (
        best_global_route,
        best_global_cost,
        total_iterations,
        successful_restarts
    )

def hill_climbing_drone(
    dist,
    start=0,
    restarts=30,
    max_iter=1000
):

    n = len(dist)

    best_global_route = None
    best_global_cost = float("inf")

    total_iterations = 0
    successful_restarts = 0

    base = list(range(n))
    base.remove(start)

    for _ in range(restarts):

        current = [
            start
        ] + random.sample(
            base,
            len(base)
        )

        current_cost = tour_cost(
            current,
            dist
        )
```

```
        improved = False

        for _ in range(max_iter):

            total_iterations += 1

            best_neighbor = current
            best_neighbor_cost = current_cost

            for i in range(1, n - 1):

                for j in range(i + 1, n):

                    neighbor = current[:]

                    neighbor[i], neighbor[j] = (
                        neighbor[j],
                        neighbor[i]
                    )

                    cost = tour_cost(
                        neighbor,
                        dist
                    )

                    if cost < best_neighbor_cost:

                        best_neighbor = neighbor
                        best_neighbor_cost = cost

            if best_neighbor_cost >= current_cost:
                break

            current = best_neighbor
            current_cost = best_neighbor_cost
            improved = True

        if improved:
            successful_restarts += 1

        if current_cost < best_global_cost:

            best_global_route = current[:]
            best_global_cost = current_cost

    return (
        best_global_route,
        best_global_cost,
        total_iterations,
        successful_restarts
    )

def backtracking_csp_drone(dist, start=0):

    n = len(dist)

    best_route = None
    best_cost = float("inf")
    expanded = 0

    def search(path, remaining, cost):

        nonlocal best_route
```

```
        nonlocal best_cost
        nonlocal expanded

        expanded += 1

        if not remaining:

            total = (
                cost
                + dist[path[-1], start]
            )

            if total < best_cost:

                best_cost = total
                best_route = path[:]

            return

        candidates = sorted(
            remaining,
            key=lambda x: dist[path[-1], x]
        )

        for nxt in candidates:

            new_cost = (
                cost
                + dist[path[-1], nxt]
            )

            if new_cost < best_cost:

                search(
                    path + [nxt],
                    remaining - {nxt},
                    new_cost
                )

    search(
        [start],
        set(range(n)) - {start},
        0.0
    )

    return (
        best_route,
        best_cost,
        expanded
    )

def branch_and_bound_drone(dist, start=0):

    n = len(dist)

    best_route = None
    best_cost = float("inf")
    expanded = 0

    def search(path, remaining, cost):

        nonlocal best_route
        nonlocal best_cost
```

```
        nonlocal expanded

        expanded += 1

        if not remaining:

            total = (
                cost
                + dist[path[-1], start]
            )

            if total < best_cost:

                best_cost = total
                best_route = path[:]

            return

        lower_bound = (
            cost
            + drone_lower_bound(
                path[-1],
                remaining,
                start,
                dist
            )
        )

        if lower_bound >= best_cost:
            return

        candidates = sorted(
            remaining,
            key=lambda x: dist[path[-1], x]
        )

        for nxt in candidates:

            new_cost = (
                cost
                + dist[path[-1], nxt]
            )

            if new_cost < best_cost:

                search(
                    path + [nxt],
                    remaining - {nxt},
                    new_cost
                )

    search(
        [start],
        set(range(n)) - {start},
        0.0
    )

    return (
        best_route,
        best_cost,
        expanded
    )

# ============================================================
```

```
# GENETIC ALGORITHM - DRONE DELIVERY
# ============================================================

def genetic_algorithm_drone(
    dist,
    start=0,
    population_size=80,
    generations=300,
    mutation_rate=0.08,
    elite_size=4
):

    n = len(dist)

    # Customer nodes excluding distribution center
    nodes = [
        i for i in range(n)
        if i != start
    ]



    # -------------------------
    # Create random route
    # -------------------------
    def create_route():

        route = [start]

        route += random.sample(
            nodes,
            len(nodes)
        )

        return route



    # -------------------------
    # Calculate fitness
    # -------------------------
    def fitness(route):

        cost = tour_cost(
            route,
            dist
        )

        return 1 / (cost + 1e-10)



    # -------------------------
    # Ordered Crossover
    # -------------------------
    def crossover(parent1, parent2):

        child = [None] * n

        child[0] = start

        left, right = sorted(
            random.sample(
                range(1, n),
                2
            )
        )
```

```
    # Copy section from parent 1
    child[left:right] = parent1[left:right]

    used = set(
        x for x in child
        if x is not None
    )

    # Fill remaining positions using parent 2
    remaining = [
        x for x in parent2
        if x not in used
    ]

    index = 0

    for i in range(1, n):

        if child[i] is None:

            child[i] = remaining[index]

            index += 1

    return child


# -------------------------
# Mutation
# -------------------------
def mutate(route):

    if random.random() < mutation_rate:

        i, j = random.sample(
            range(1, n),
            2
        )

        route[i], route[j] = (
            route[j],
            route[i]
        )


# -------------------------
# Initial population
# -------------------------
population = [
    create_route()
    for _ in range(
        population_size
    )
]


# Best solution
best_route = min(
    population,
    key=lambda route:
        tour_cost(
            route,
```

```
            dist
        )
)



best_cost = tour_cost(
    best_route,
    dist
)



# Store convergence history
history = [
    best_cost
]



# ========================================================
# GENERATIONS
# ========================================================

for generation in range(
    generations
):

    # Calculate fitness
    fitness_values = [
        fitness(route)
        for route in population
    ]



    # Sort population according to fitness
    ranked_population = sorted(
        population,
        key=lambda route:
            fitness(route),
        reverse=True
    )



    # Keep elite solutions
    new_population = [
        route[:]
        for route in
        ranked_population[
            :elite_size
        ]
    ]



    # Generate new population
    while len(new_population) < population_size:

        # Tournament selection
        candidates = random.sample(
            population,
            4
        )



        parent1 = max(
            candidates,
            key=lambda route:
```

```
                fitness(route)
        )


        candidates = random.sample(
            population,
            4
        )


        parent2 = max(
            candidates,
            key=lambda route:
                fitness(route)
        )


        # Crossover
        child = crossover(
            parent1,
            parent2
        )


        # Mutation
        mutate(child)


        new_population.append(
            child
        )


    population = new_population


    # Current best
    current_best = min(
        population,
        key=lambda route:
            tour_cost(
                route,
                dist
            )
    )


    current_cost = tour_cost(
        current_best,
        dist
    )


    # Update global best
    if current_cost < best_cost:

        best_route = (
            current_best[:]
        )

        best_cost = (
            current_cost
        )
```

```
        history.append(
            best_cost
        )


    return (
        best_route,
        best_cost,
        generations,
        history
    )

results = []

start_node = 0

# -------------------------
# BFS
# -------------------------
tracemalloc.start()
t0 = time.perf_counter()

route, cost, expanded = bfs_drone(DIST, start_node)

time_taken = time.perf_counter() - t0
current, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

results.append({
    "Algorithm": "BFS",
    "Route": route_text(route),
    "Distance": cost,
    "Time (s)": time_taken,
    "Peak Memory (MB)": peak / 1024 / 1024,
    "States Expanded": expanded,
    "Iterations": np.nan,
    "Generations": np.nan,
    "Success / Restarts": np.nan
})



# -------------------------
# DFS
# -------------------------
tracemalloc.start()
t0 = time.perf_counter()

route, cost, expanded = dfs_drone(DIST, start_node)

time_taken = time.perf_counter() - t0
current, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

results.append({
    "Algorithm": "DFS",
    "Route": route_text(route),
    "Distance": cost,
    "Time (s)": time_taken,
    "Peak Memory (MB)": peak / 1024 / 1024,
    "States Expanded": expanded,
    "Iterations": np.nan,
    "Generations": np.nan,
```

```
    "Success / Restarts": np.nan
})



# -------------------------
# Uniform Cost Search
# -------------------------
tracemalloc.start()
t0 = time.perf_counter()

route, cost, expanded = ucs_drone(DIST, start_node)

time_taken = time.perf_counter() - t0
current, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

results.append({
    "Algorithm": "Uniform Cost Search",
    "Route": route_text(route),
    "Distance": cost,
    "Time (s)": time_taken,
    "Peak Memory (MB)": peak / 1024 / 1024,
    "States Expanded": expanded,
    "Iterations": np.nan,
    "Generations": np.nan,
    "Success / Restarts": np.nan
})



# -------------------------
# A*
# -------------------------
tracemalloc.start()
t0 = time.perf_counter()

route, cost, expanded = astar_drone(DIST, start_node)

time_taken = time.perf_counter() - t0
current, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

results.append({
    "Algorithm": "A*",
    "Route": route_text(route),
    "Distance": cost,
    "Time (s)": time_taken,
    "Peak Memory (MB)": peak / 1024 / 1024,
    "States Expanded": expanded,
    "Iterations": np.nan,
    "Generations": np.nan,
    "Success / Restarts": np.nan
})



# -------------------------
# Greedy Best-First Search
# -------------------------
tracemalloc.start()
t0 = time.perf_counter()

route, cost, expanded = greedy_drone(DIST, start_node)

time_taken = time.perf_counter() - t0
current, peak = tracemalloc.get_traced_memory()
```

```
tracemalloc.stop()

results.append({
    "Algorithm": "Greedy Best-First",
    "Route": route_text(route),
    "Distance": cost,
    "Time (s)": time_taken,
    "Peak Memory (MB)": peak / 1024 / 1024,
    "States Expanded": expanded,
    "Iterations": np.nan,
    "Generations": np.nan,
    "Success / Restarts": np.nan
})



# -------------------------
# Hill Climbing
# -------------------------
tracemalloc.start()
t0 = time.perf_counter()

route, cost, iterations, restarts = hill_climbing_drone(
    DIST,
    start_node
)

time_taken = time.perf_counter() - t0
current, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

results.append({
    "Algorithm": "Hill Climbing",
    "Route": route_text(route),
    "Distance": cost,
    "Time (s)": time_taken,
    "Peak Memory (MB)": peak / 1024 / 1024,
    "States Expanded": np.nan,
    "Iterations": iterations,
    "Generations": np.nan,
    "Success / Restarts": restarts
})



# -------------------------
# Genetic Algorithm
# -------------------------
tracemalloc.start()
t0 = time.perf_counter()

route, cost, generations, history = genetic_algorithm_drone(
    DIST,
    start_node
)

time_taken = time.perf_counter() - t0
current, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

results.append({
    "Algorithm": "Genetic Algorithm",
    "Route": route_text(route),
    "Distance": cost,
    "Time (s)": time_taken,
    "Peak Memory (MB)": peak / 1024 / 1024,
    "States Expanded": np.nan,
```

```
    "Iterations": np.nan,
    "Generations": generations,
    "Success / Restarts": np.nan
})



# -------------------------
# CSP Backtracking + Pruning
# -------------------------
tracemalloc.start()
t0 = time.perf_counter()

route, cost, expanded = backtracking_csp_drone(
    DIST,
    start_node
)

time_taken = time.perf_counter() - t0
current, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

results.append({
    "Algorithm": "CSP Backtracking + Pruning",
    "Route": route_text(route),
    "Distance": cost,
    "Time (s)": time_taken,
    "Peak Memory (MB)": peak / 1024 / 1024,
    "States Expanded": expanded,
    "Iterations": np.nan,
    "Generations": np.nan,
    "Success / Restarts": np.nan
})



# -------------------------
# CSP Branch and Bound
# -------------------------
tracemalloc.start()
t0 = time.perf_counter()

route, cost, expanded = branch_and_bound_drone(
    DIST,
    start_node
)

time_taken = time.perf_counter() - t0
current, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

results.append({
    "Algorithm": "CSP Branch & Bound",
    "Route": route_text(route),
    "Distance": cost,
    "Time (s)": time_taken,
    "Peak Memory (MB)": peak / 1024 / 1024,
    "States Expanded": expanded,
    "Iterations": np.nan,
    "Generations": np.nan,
    "Success / Restarts": np.nan
})



# -------------------------
# Final Results
# -------------------------
```

```
results_df = pd.DataFrame(results)

results_df

optimal_distance = results_df["Distance"].min()

results_df["Optimality Gap (%)"] = (
    (results_df["Distance"] - optimal_distance)
    / optimal_distance
    * 100
)

results_df["Distance"] = results_df["Distance"].round(2)
results_df["Time (s)"] = results_df["Time (s)"].round(6)
results_df["Peak Memory (MB)"] = (
    results_df["Peak Memory (MB)"].round(3)
)
results_df["Optimality Gap (%)"] = (
    results_df["Optimality Gap (%)"].round(3)
)

display(results_df)

plt.figure(figsize=(8, 5))

plt.plot(
    history,
    linewidth=2
)

plt.title(
    "Genetic Algorithm Convergence - Drone Delivery"
)

plt.xlabel("Generation")
plt.ylabel("Best Route Distance")

plt.grid(alpha=0.25)
plt.show()

plt.figure(figsize=(10, 5))

plt.bar(
    results_df["Algorithm"],
    results_df["Distance"]
)

plt.title(
    "Route Distance Comparison"
)

plt.xlabel("Algorithm")
plt.ylabel("Total Distance")

plt.xticks(
    rotation=45,
    ha="right"
)

plt.grid(
    axis="y",
    alpha=0.25
)
```

```
plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 5))

plt.bar(
    results_df["Algorithm"],
    results_df["Time (s)"]
)

plt.title(
    "Execution Time Comparison"
)

plt.xlabel("Algorithm")
plt.ylabel("Time (seconds)")

plt.xticks(
    rotation=45,
    ha="right"
)

plt.grid(
    axis="y",
    alpha=0.25
)

plt.tight_layout()
plt.show()

print("""
1. BFS, DFS, UCS, A*, Hill Climbing, GA and CSP found the shortest route of 283.84 km.
2. Greedy Best-First produced 412.37 km with a 45.283% optimality gap.
3. BFS, DFS and UCS expanded 986410 states each.
4. A* expanded 16 states, while CSP Branch & Bound expanded 79 states.
5. Hill Climbing used 177 iterations and 30 restarts, while GA used 300 generations.
6. A* and CSP Branch & Bound had very low execution times.
7. BFS used the highest peak memory of approximately 75.251 MB.
8. CSP methods use constraints, while Branch & Bound adds cost-based pruning.
9. Overall, informed and constraint-based methods reduce unnecessary search effort.
""")
```

6. RESULT ANALYSIS

- BFS, DFS, UCS, A*, Hill Climbing, GA, and CSP Backtracking / Branch & Bound all converge to the same optimal route distance of 283.84 km, confirming the correctness of each implementation.
- Greedy Best-First Search sacrifices optimality for speed, producing a route 45.28% longer than the optimal, since it only considers the heuristic estimate and ignores the cost already incurred.
- Uninformed methods (BFS, DFS, UCS) expand an extremely large number of states (986,410 each) because they explore the search space blindly, without any heuristic guidance.
- A* expands only 16 states to reach the optimal solution, demonstrating the power of combining actual cost with an admissible heuristic (MST-based lower bound).
- CSP Branch and Bound expands 79 states — far fewer than plain backtracking would — because cost-based pruning eliminates unpromising branches early.
- Hill Climbing and the Genetic Algorithm both reach the optimal distance through iterative improvement (177 iterations / 30 restarts, and 300 generations respectively), showing that metaheuristics can match exact methods on small problem instances.
- BFS records the highest peak memory usage (≈75.25 MB), since it must store every partial route at the current search depth simultaneously.
- Overall, informed search (A*) and constraint-based pruning (Branch & Bound) are the most efficient strategies, achieving the optimal route with minimal computational effort compared to uninformed search.

7. RESULT

The drone delivery route optimization problem was successfully modeled as a TSP instance and solved using nine different search algorithms. BFS, DFS, UCS, A*, Hill Climbing, the Genetic Algorithm, and both CSP-based methods all obtained the optimal route distance of 283.84 km, while Greedy Best-First Search converged to a suboptimal route. Performance comparison in terms of execution time, memory usage, and states expanded clearly demonstrates the computational advantage of informed and constraint-based search over uninformed search.

8. CONCLUSION

This experiment demonstrates how different search paradigms — uninformed, informed, local search/metaheuristic, and constraint-based — can be applied to solve a real-world route optimization problem. While all optimal-guaranteeing methods (BFS, DFS, UCS, A*, CSP Backtracking, Branch & Bound) reached the same best distance, they differed drastically in computational cost, with A* and Branch & Bound proving to be the most efficient due to their use of heuristic guidance and pruning. Metaheuristic approaches such as Hill Climbing and Genetic Algorithms offer a practical alternative for larger, real-world delivery networks where exact methods become computationally infeasible. Overall, the experiment highlights the trade-off between solution optimality and computational efficiency that must be considered when selecting a search strategy for AI-based logistics and route planning applications.